multiverse-machine-SRD

The Multiverse Machine — Software Requirements Document

One line: Type a sentence, watch an AI write it word by word, and see every word it *almost* wrote branch off as faded parallel timelines you can play with.

---

1. Functional Requirements

Feature	Functional Requirement
Prompt input	User types a starting sentence and presses Generate. Empty input is blocked with a gentle nudge. A few example prompts are offered as one-click starters.
Token-by-token generation	The model continues the sentence one token at a time, rendered live so the user watches the "chosen" text appear word by word rather than all at once.
Ghost branches	At each generated token, the top alternative tokens the model considered are shown as faded branches splitting off from the main line. The chosen token stays solid and bright; the rest fade by how unlikely they were.
Probability labels	Each branch (chosen and ghost) shows how likely the model thought it was, as a clean percentage. This is the "confidence made visible" layer.
Temperature slider	A single hero control. Dragging it low collapses the multiverse toward one predictable line; dragging it high explodes the branching. The visual updates the *next* generation live so the effect is felt, not explained.
Branch count control	User can set how many ghost alternatives appear per token (e.g. 3 to 8), trading clarity for chaos.
Timeline switching	Clicking any ghost token rewinds reality to that point, makes that branch the new "chosen" path, and continues generating from there. The old path fades into the background as an abandoned timeline.
In-browser model	The language model runs entirely in the user's browser. No sign-in, no API key, no data leaves the device.
Model loading state	On first visit the model downloads once and caches. A clear, on-brand loading state shows progress so the wait feels intentional, not broken.
Reset / replay	User can clear the canvas and start a new sentence, or replay the current multiverse from the beginning.
Share	User can share their multiverse — a link that reopens the same prompt and settings, plus a clean image/screenshot export for LinkedIn. This is the growth engine, not an afterthought.

2. Non-Functional Requirements

- **Zero running cost.** Fully client-side. Traffic and virality must never generate a server or API bill.

- **Performance.** Token generation and branch animation stay smooth and responsive on a normal laptop. Generation is fast enough that the "watch it write" effect feels alive, not laggy.
- **Visual craft.** The look and motion are a first-class requirement, not polish added later. This is the entire differentiator versus existing developer tools that do the same thing ugly.
- **First-load experience.** The one-time model download is communicated honestly and made to feel part of the experience. Model kept small to keep this download reasonable.
- **Browser support.** Targets WebGPU-capable browsers (Chrome, Edge) as primary. Graceful, clearly-worded fallback message where WebGPU is unavailable rather than a silent failure.
- **Reduced-motion mode.** Because the whole thing is animation, an accessible mode that tones down movement for users who need it.
- **Privacy as a feature.** "Nothing you type ever leaves your browser" is stated plainly in the UI — it's both true and a shareable hook.
- **Responsive layout.** Usable and beautiful on desktop first (where WebGPU is strongest); mobile shows a tasteful "best on desktop" state rather than a broken one.

3. Tech Stack

Layer	Choice	Why
Framework	React + TypeScript + Vite	Fast, familiar to you, and the ecosystem for the visualization libraries below.
Model inference	Transformers.js v4 (`@huggingface/transformers`) on WebGPU	Runs a real model in-browser with no server or key, and gives full access to the probability distribution at every token — which is the raw material of the whole project.
Model	A deliberately *small* instruct model (e.g. Llama-3.2-1B-Instruct, with Qwen2.5-0.5B as a faster fallback)	Small models are more uncertain, so they branch more dramatically. The "wow" comes from a small model, and it keeps the download light.
Visualization	D3.js (or SVG + a motion library) for the animated branching tree	Purpose-built for animated node-link trees; handles the growing, forking, collapsing multiverse cleanly.
Styling	Tailwind CSS	Fast iteration on the look, which here is the product.
Hosting	Static host (Vercel / Netlify / GitHub Pages)	No backend to run, secure, and free to serve.
Optional stretch	"Power mode": bring-your-own-key OpenAI-compatible API for a bigger/faster model	For power users who want more muscle, without ever making it a cost you carry.

Note on the pivotal decision: this design runs the model in the browser rather than through Claude's or anyone's API. Claude's API doesn't expose per-token alternatives at all, and even the APIs that do would charge per visitor and cap you at the top few alternatives. Running a small model locally is free at any scale, exposes the full distribution, and — because small models are gloriously uncertain — produces a far better multiverse.